

Node.js 교과서



Node.js 교과서 개정2판

9장

9.1 프로젝트 구조 갖추기

9.2 데이터베이스 세팅하기

9.3 Passport 모듈로 로그인

9.4 Multer 모듈로 이미지 업로드

9.5 프로젝트 마무리하기

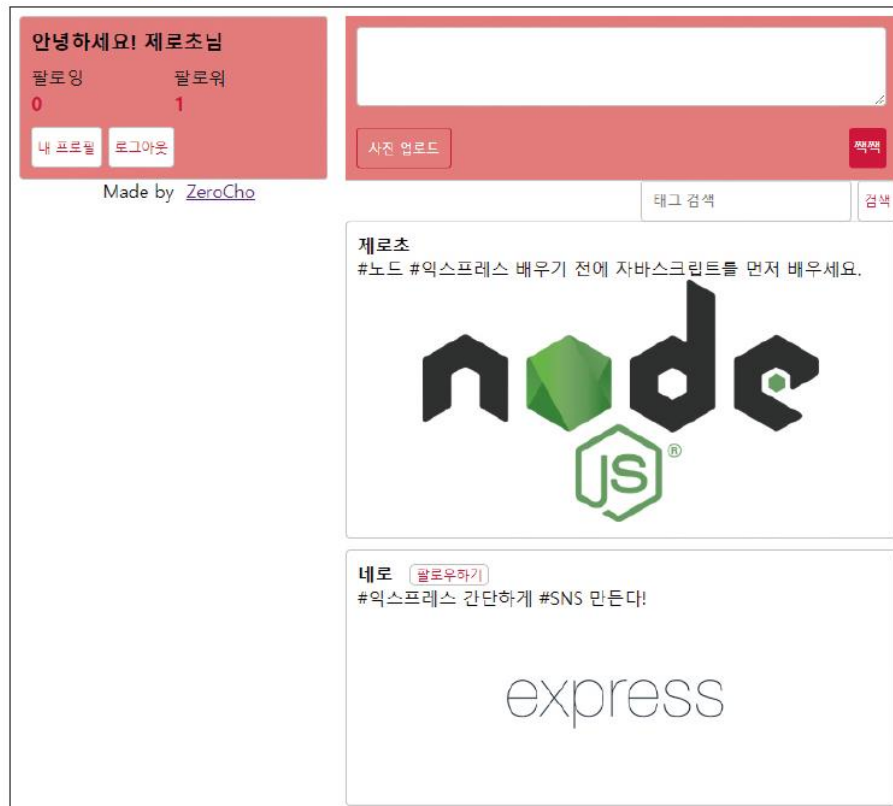
9.1 프로젝트 구조 갖추기

1. NodeBird SNS 서비스

» 기능: 로그인, 이미지 업로드, 게시물 작성, 해시태그 검색, 팔로잉

- express-generator 대신 직접 구조를 갖추
- 프론트엔드 코드보다 노드 라우터 중심으로 볼 것
- 관계형 데이터베이스 MySQL 선택

♥ 그림 9-1 NodeBird 완성 화면





2. 프로젝트 시작하기

» nodebird 폴더를 만들고 package.json 파일 생성

- 노드 프로젝트의 기본

package.json

```
{
  "name": "nodebird",
  "version": "0.0.1",
  "description": "익스프레스로 만드는 SNS 서비스",
  "main": "app.js",
  "scripts": {
    "start": "nodemon app"
  },
  "author": "ZeroCho",
  "license": "MIT"
}
```

» 시퀀라이즈 폴더 구조 생성

콘솔

```
$ npm i sequelize mysql2 sequelize-cli
$ npx sequelize init
```

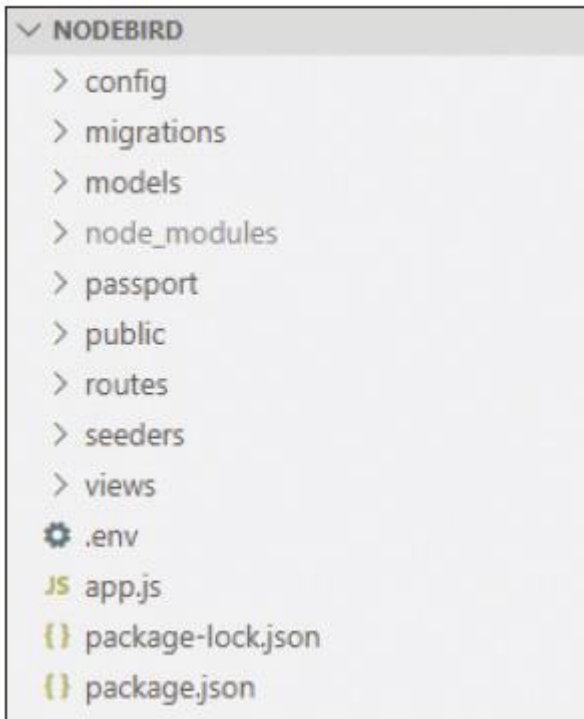


3. 폴더 구조 설정

» views(템플릿 엔진), routes(라우터), public(정적 파일), passport(패스포트) 폴더 생성

- app.js와 .env 파일도 생성

▼ 그림 9-2 nodebird 폴더 구조





4. 패키지 설치와 nodemon

» npm 패키지 설치 후 nodemon도 설치

- nodemon은 서버 코드가 변경되었을 때 자동으로 서버를 재시작해줌
- nodemon은 콘솔 명령어이기 때문에 글로벌로 설치

콘솔

```
$ npm i express cookie-parser express-session morgan multer dotenv nunjucks
```

```
$ npm i -D nodemon
```



5. app.js

» 노드 서버의 핵심인 app.js 파일 작성

- 소스 코드는

<https://github.com/zerocho/nodejs-book>

- .env도 같이 추가

.env

COOKIE_SECRET=cookiesecret

```
app.js
const express = require('express');
const cookieParser = require('cookie-parser');
const morgan = require('morgan');
const path = require('path');
const session = require('express-session');
const nunjucks = require('nunjucks');
const dotenv = require('dotenv');

dotenv.config();
const pageRouter = require('./routes/page');

const app = express();
app.set('port', process.env.PORT || 8081);
app.set('view engine', 'html');
nunjucks.configure('views', {
  express: app,
  watch: true,
});

app.use(morgan('dev'));
app.use(express.static(path.join(__dirname, 'public')));
app.use(express.json());
app.use(express.urlencoded({ extended: false }));
app.use(cookieParser(process.env.COOKIE_SECRET));
app.use(session({
  resave: false,
  saveUninitialized: false,
  secret: process.env.COOKIE_SECRET,
  cookie: {
    httpOnly: true,
    secure: false,
  },
}));

app.use('/', pageRouter);

app.use((req, res, next) => {
  const error = new Error(`${req.method} ${req.url} 라우터가 없습니다.`);
  error.status = 404;
  next(error);
});

app.use((err, req, res, next) => {
  res.locals.message = err.message;
  res.locals.error = process.env.NODE_ENV !== 'production' ? err : {};
  res.status(err.status || 500);
  res.render('error');
});

app.listen(app.get('port'), () => {
  console.log(app.get('port'), '번 포트에서 대기 중');
});
```


6. 라우터 생성

» 소스 코드는 <https://github.com/ZeroCho/nodejs-book/tree/master/ch9/9.1/nodebird>

- routes/page.js: 템플릿 엔진을 렌더링하는 라우터
- views/layout.html: 프론트 엔드 화면 레이아웃(로그인/유저 정보 화면)
- views/main.html: 메인 화면(게시글들이 보임)
- views/profile.html: 프로필 화면(팔로잉 관계가 보임)
- views/error.html: 에러 발생 시 에러가 표시될 화면
- public/main.css: 화면 CSS

» npm start로 서버 실행 후 <http://localhost:8001> 접속

▼ 그림 9-3 NodeBird 메인 화면



The image shows a web form for NodeBird. It has a red header bar. Below the header, there are two input fields for '이메일' (Email) and '비밀번호' (Password). Below these fields are three buttons: '로그인' (Login), '카카오' (Kakao), and '회원가입' (Sign Up). At the bottom of the form, it says 'Made by ZeroCho'. To the right of the form, there is a search bar with the text '태그 검색' (Tag Search) and a '검색' (Search) button.

9.2 데이터베이스 세팅하기

1. 모델 생성

» 소스 코드는 <https://github.com/ZeroCho/nodejs-book/tree/master/ch9/9.1/nodebird>

- models/user.js: 사용자 테이블과 연결됨
 - provider: 카카오 로그인인 경우 kakao, 로컬 로그인(이메일/비밀번호)인 경우 local
 - snsId: 카카오 로그인인 경우 주어지는 id
- models/post.js: 게시글 내용과 이미지 경로를 저장(이미지는 파일로 저장)
- models/hashtag.js: 해시태그 이름을 저장(나중에 태그로 검색하기 위해서)



2. models/index.js

» 시퀀라이즈가 자동으로 생성해주는 코드 대신 다음과 같이 변경

- 모델들을 불러옴(require)
- 모델 간 관계가 있는 경우 관계 설정
- User(1):Post(다)
- Post(다):Hashtag(다)
- User(다):User(다)

models/user.js

```
...
static associate(db) {
  db.User.hasMany(db.Post);
  db.User.belongsToMany(db.User, {
    foreignKey: 'followingId',
    as: 'Followers',
    through: 'Follow',
  });
  db.User.belongsToMany(db.User, {
    foreignKey: 'followerId',
    as: 'Followings',
    through: 'Follow',
  });
}
...
```

models/index.js

```
const Sequelize = require('sequelize');
const env = process.env.NODE_ENV || 'development';
const config = require('../config/config')[env];
const User = require('./user');
const Post = require('./post');
const Hashtag = require('./hashtag');

const db = {};
const sequelize = new Sequelize(
  config.database, config.username, config.password, config,
);

db.sequelize = sequelize;
db.User = User;
db.Post = Post;
db.Hashtag = Hashtag;

User.init(sequelize);
Post.init(sequelize);
Hashtag.init(sequelize);

User.associate(db);
Post.associate(db);
Hashtag.associate(db);

module.exports = db;
```



3. associate 작성하기

» 모델간의 관계들 associate에 작성

- 1대다: hasMany와 belongsTo
- 다대다: belongsToMany
 - foreignKey: 외래키
 - as: 컬럼에 대한 별명
 - through: 중간 테이블명

models/user.js

```
...
static associate(db) {
  db.User.hasMany(db.Post);
  db.User.belongsToMany(db.User, {
    foreignKey: 'followingId',
    as: 'Followers',
    through: 'Follow',
  });
  db.User.belongsToMany(db.User, {
    foreignKey: 'followerId',
    as: 'Followings',
    through: 'Follow',
  });
}
...
```

models/hashtag.js

```
...
static associate(db) {
  db.Hashtag.belongsToMany(db.Post, { through: 'PostHashtag' });
}
};
```

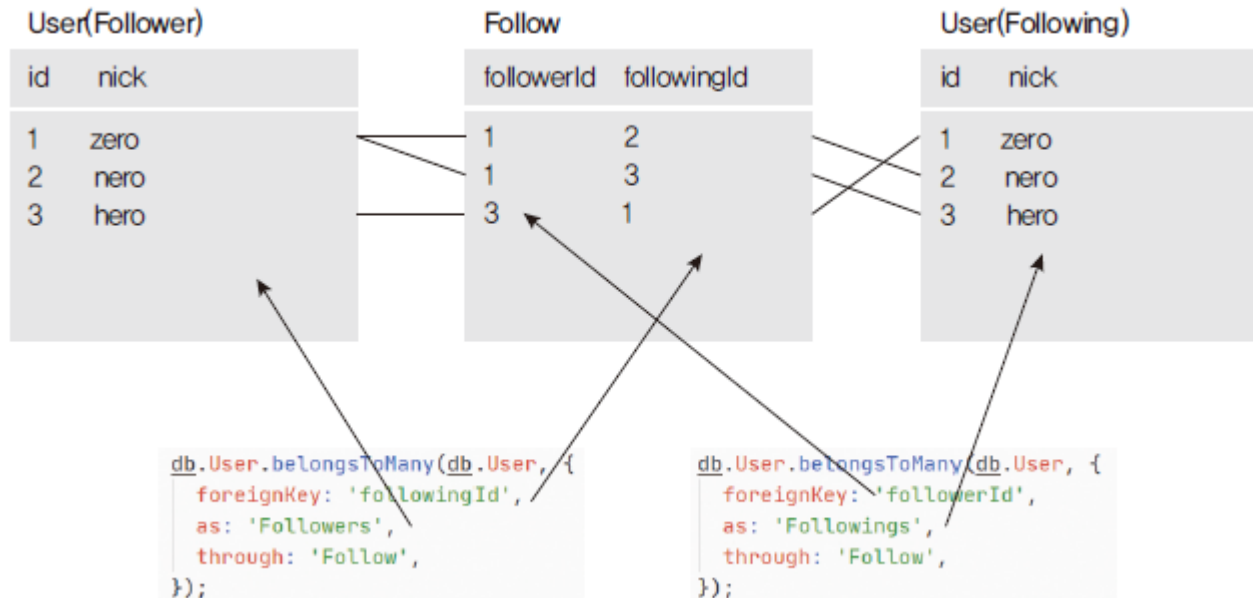
models/post.js

```
...
static associate(db) {
  db.Post.belongsTo(db.User);
  db.Post.belongsToMany(db.Hashtag, { through: 'PostHashtag' });
}
...
```

4. 팔로잉-팔로워 다대다 관계

» User(다):User(다)

- 다대다 관계이므로 중간 테이블(Follow) 생성됨
- 모델 이름이 같으므로 구분 필요함(as가 구분자 역할, foreignKey는 반대 테이블 컬럼의 프라이머리 키 컬럼)
- 시퀀라이즈는 as 이름을 바탕으로 자동으로 addFollower, getFollowers, addFollowing, getFollowings 메서드 생성



5. 시퀀라이즈 설정하기

» 시퀀라이즈 설정은 config/config.json에서

- 개발환경용 설정은 development 아래에

config/config.json

```
{
  "development": {
    "username": "root",
    "password": "[root 비밀번호]",
    "database": "nodebird",
    "host": "127.0.0.1",
    "dialect": "mysql"
  },
}
```

» 설정 파일 작성 후 nodebird 데이터베이스 생성

콘솔

```
$ npx sequelize db:create
Sequelize CLI [Node: 14.0.0, CLI: 6.1.0, ORM: 6.2.3]

Loaded configuration file "config\config.json".
Using environment "development".
Database nodebird created.
```



6. 모델과 서버 연결하기

» sequelize.sync()가 테이블 생성

- IF NOT EXIST(SQL문)으로 테이블이 없을 때만 생성해줌

app.js

```
...
const pageRouter = require('./routes/page');
const { sequelize } = require('./models');
...
nunjucks.configure('views', {
  express: app,
  watch: true,
});
sequelize.sync({ force: false })
  .then(() => {
    console.log('데이터베이스 연결 성공');
  })
  .catch((err) => {
    console.error(err);
  });

app.use(morgan('dev'));
...
```


6. 모델과 서버 연결하기

» npm start로 서버 실행 시 콘솔에 SQL문이 표시됨

콘솔

```
$ npm start
> nodebird@0.0.1 start C:\Users\zerocho\nodebird
> nodemon app

[nodemon] 2.0.2
[nodemon] to restart at any time, enter `rs`
[nodemon] watching dir(s): *.*
[nodemon] watching extensions: js,mjs,json
[nodemon] starting `node app.js`
8001 번 포트에서 대기 중
Executing (default): CREATE TABLE IF NOT EXISTS `users` (`id` INTEGER NOT NULL auto_increment , `title` VARCHAR(15) NOT NULL UNIQUE, `createdAt` DATETIME NOT NULL, `updatedAt` DATETIME NOT NULL, PRIMARY KEY (`id`)) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE utf8mb4_general_ci;
...
데이터베이스 연결 성공
```

9.3 Passport 모듈로 로그인

1. 패스포트 설치하기

» 로그인 과정을 쉽게 처리할 수 있게 도와주는 Passport 설치하기

- 비밀번호 암호화를 위한 bcrypt도 같이 설치
- 설치 후 app.js와도 연결
- passport.initialize(): 요청 객체에 passport 설정을 심음
- passport.session(): req.session 객체에 passport 정보를 저장
- express-session 미들웨어에 의존하므로 이보다 더 뒤에 위치해야 함

콘솔

```
$ npm i passport passport-local passport-kakao bcrypt
```

app.js

```
...
const dotenv = require('dotenv');
const passport = require('passport');

dotenv.config();
const pageRouter = require('./routes/page');
const { sequelize } = require('./models');
const passportConfig = require('./passport');

const app = express();
passportConfig(); // 패스포트 설정
app.set('port', process.env.PORT || 8001);
app.set('view engine', 'html');

...
app.use(session({
  resave: false,
  saveUninitialized: false,
  secret: process.env.COOKIE_SECRET,
  cookie: {
    httpOnly: true,
    secure: false,
  },
}));
app.use(passport.initialize());
app.use(passport.session());

app.use('/', pageRouter);
```



2. 패스포트 모듈 작성

» passport/index.js 작성

- passport.serializeUser: req.session 객체에 어떤 데이터를 저장할 지 선택, 사용자 정보를 다 들고 있으면 메모리를 많이 차지하기 때문에 사용자의 아이디만 저장
- passport.deserializeUser: req.session에 저장된 사용자 아이디를 바탕으로 DB 조회로 사용자 정보를 얻어낸 후 req.user에 저장

로그인 사용자 정보



```
passport.serializeUser((user, done) => {
  done(null, user.id);
});
```

```
passport.deserializeUser((id, done) => {
  User.findOne({ where: { id } })
    .then(user => done(null, user))
    .catch(err => done(err));
});
```

req.user에 저장

passport/index.js

```
const passport = require('passport');
const local = require('./localStrategy');
const kakao = require('./kakaoStrategy');
const User = require('../models/user');

module.exports = () => {
  passport.serializeUser((user, done) => {
    done(null, user.id);
  });

  passport.deserializeUser((id, done) => {
    User.findOne({ where: { id } })
      .then(user => done(null, user))
      .catch(err => done(err));
  });

  local();
  kakao();
};
```

3. 패스포트 처리 과정

» 로그인 과정

- 1. 로그인 요청이 들어옴
- 2. passport.authenticate 메서드 호출
- 3. 로그인 전략 수행(전략은 뒤에 알아봄)
- 4. 로그인 성공 시 사용자 정보 객체와 함께 req.login 호출
- 5. req.login 메서드가 passport.serializeUser 호출
- 6. req.session에 사용자 아이디만 저장
- 7. 로그인 완료

» 로그인 이후 과정

- 1. 모든 요청에 passport.session() 미들웨어가 passport.deserializeUser 메서드 호출
- 2. req.session에 저장된 아이디로 데이터베이스에서 사용자 조회
- 3. 조회된 사용자 정보를 req.user에 저장
- 4. 라우터에서 req.user 객체 사용 가능



4. 로컬 로그인 구현하기

» passport-local 패키지 필요

- 로컬 로그인 전략 수립
- 로그인에만 해당하는 전략이므로 회원가입은 따로 만들어야 함
- 사용자가 로그인했는지, 하지 않았는지 여부를 체크하는 미들웨어도 만들

routes/middlewares.js

```
exports.isLoggedIn = (req, res, next) => {
  if (req.isAuthenticated()) {
    next();
  } else {
    res.status(403).send('로그인 필요');
  }
};

exports.isNotLoggedIn = (req, res, next) => {
  if (!req.isAuthenticated()) {
    next();
  } else {
    const message = encodeURIComponent('로그인한 상태입니다.');
```

routes/page.js

```
const express = require('express');
const { isLoggedIn, isNotLoggedIn } = require('./middlewares');

const router = express.Router();

router.use((req, res, next) => {
  res.locals.user = req.user;
  res.locals.followerCount = 0;
  res.locals.followingCount = 0;
  res.locals.followerIdList = [];
  next();
});

router.get('/profile', isLoggedIn, (req, res) => {
  res.render('profile', { title: '내 정보 - NodeBird' });
});

router.get('/join', isNotLoggedIn, (req, res) => {
  res.render('join', { title: '회원가입 - NodeBird' });
});

...
```



5. 회원가입 라우터

» routes/auth.js 작성

- bcrypt.hash로 비밀번호 암호화
- hash의 두 번째 인수는 암호화 라운드
- 라운드가 높을수록 안전하지만 오래 걸
- 적당한 라운드를 찾는 게 좋음
- ?error 쿼리스트링으로 1회성 메시지

routes/auth.js

```
const express = require('express');
const passport = require('passport');
const bcrypt = require('bcrypt');
const { isLoggedIn, isNotLoggedIn } = require('./middlewares');
const User = require('../models/user');

const router = express.Router();

router.post('/join', isNotLoggedIn, async (req, res, next) => {
  const { email, nick, password } = req.body;
  try {
    const exUser = await User.findOne({ where: { email } });
    if (exUser) {
      return res.redirect('/join?error=exist');
    }
    const hash = await bcrypt.hash(password, 12);
    await User.create({
      email,
      nick,
      password: hash,
    });
    return res.redirect('/');
  } catch (error) {
    console.error(error);
    return next(error);
  }
});
```



6. 로그인 라우터

» routes/auth.js 작성

- passport.authenticate('local'): 로컬 전략
- 전략을 수행하고 나면 authenticate의 콜백 함수 호출됨
- authError: 인증 과정 중 에러,
- user: 인증 성공 시 유저 정보
- info: 인증 오류에 대한 메시지
- 인증이 성공했다면 req.login으로 세션에 유저 정보 저장

```
router.post('/login', isLoggedIn, (req, res, next) => {
  passport.authenticate('local', (authError, user, info) => {
    if (authError) {
      console.error(authError);
      return next(authError);
    }
    if (!user) {
      return res.redirect(`/?loginError=${info.message}`);
    }
    return req.login(user, (loginError) => {
      if (loginError) {
        console.error(loginError);
        return next(loginError);
      }
      return res.redirect('/');
    });
  })(req, res, next); // 미들웨어 내의 미들웨어에는 (req, res, next)를 붙입니다.
});
```

```
router.get('/logout', isLoggedIn, (req, res) => {
  req.logout();
  req.session.destroy();
  res.redirect('/');
});
```

```
module.exports = router;
```




7. 로컬 전략 작성

» passport/localStrategy.js 작성

- usernameField와 passwordField가 input 태그의 name(body-parser의 req.body)
- 사용자가 DB에 저장되어있는지 확인한 후 있다면 비밀번호 비교(bcrypt.compare)
- 비밀번호까지 일치한다면 로그인

♥ 그림 9-9 done과 authenticate의 관계

1. 로그인 성공 시

```
done(null, exUser);
↓ ↓
passport.authenticate('local', (authError, user, info) => {
```

2. 로그인 실패 시

```
done(null, false, { message: '비밀번호가 일치하지 않습니다.' });
↓ ↓ ↓
passport.authenticate('local', (authError, user, info) => {
```

3. 서버 에러 시

```
done(error);
↓
passport.authenticate('local', (authError, user, info) => {
```

passport/localStrategy.js

```
const passport = require('passport');
const LocalStrategy = require('passport-local').Strategy;
const bcrypt = require('bcrypt');

const User = require('../models/user');

module.exports = () => {
  passport.use(new LocalStrategy({
    usernameField: 'email',
    passwordField: 'password',
  }, async (email, password, done) => {
    try {
      const exUser = await User.findOne({ where: { email } });
      if (exUser) {
        const result = await bcrypt.compare(password, exUser.password);
        if (result) {
          done(null, exUser);
        } else {
          done(null, false, { message: '비밀번호가 일치하지 않습니다.' });
        }
      } else {
        done(null, false, { message: '가입되지 않은 회원입니다.' });
      }
    } catch (error) {
      console.error(error);
      done(error);
    }
  }));
};
```



8. 카카오 로그인 구현

» passport/kakaoStrategy.js 작성

- clientID에 카카오 앱 아이디 추가
- callbackURL: 카카오 로그인 후 카카오가 결과를 전송해줄 URL
- accessToken, refreshToken: 로그인 성공 후 카카오가 보내준 토큰(사용하지 않음)
- profile: 카카오가 보내준 유저 정보
- profile의 정보를 바탕으로 회원가입

passport/kakaoStrategy.js

```
const passport = require('passport');
const KakaoStrategy = require('passport-kakao').Strategy;

const User = require('../models/user');

module.exports = () => {
  passport.use(new KakaoStrategy({
    clientID: process.env.KAKAO_ID,
    callbackURL: '/auth/kakao/callback',
  }, async (accessToken, refreshToken, profile, done) => {
    console.log('kakao profile', profile);
    try {
      const exUser = await User.findOne({
        where: { snsId: profile.id, provider: 'kakao' },
      });
      if (exUser) {
        done(null, exUser);
      } else {
        const newUser = await User.create({
          email: profile._json && profile._json.kaccount_email,
          nick: profile.displayName,
          snsId: profile.id,
          provider: 'kakao',
        });
        done(null, newUser);
      }
    } catch (error) {
      console.error(error);
      done(error);
    }
  }));
};
```

9. 카카오 로그인용 라우터 만들기

» 회원가입과 로그인이 전략에서 동시에 수행됨

- passport.authenticate('kakao')만 하면 됨
- /kakao/callback 라우터에서는 인증 성공 시(res.redirect)와 실패 시(failureRedirect) 리다이렉트할 경로 지정

routes/auth.js

```
...
router.get('/logout', isLoggedIn, (req, res) => {
  req.logout();
  req.session.destroy();
  res.redirect('/');
});

router.get('/kakao', passport.authenticate('kakao'));

router.get('/kakao/callback', passport.authenticate('kakao', {
  failureRedirect: '/',
}), (req, res) => {
  res.redirect('/');
});

module.exports = router;
```

app.js

```
...
const pageRouter = require('./routes/page');
const authRouter = require('./routes/auth');
const { sequelize } = require('./models');
...
app.use('/', pageRouter);
app.use('/auth', authRouter);
...
```



10. 카카오 로그인 앱 만들기

» <https://developers.kakao.com>에 접속하여 회원가입

- NodeBird 앱 만들기

▼ 그림 9-10 애플리케이션 추가하기 버튼 클릭



▼ 그림 9-11 앱 정보 작성 화면

애플리케이션 추가

앱 아이콘

이미지 업로드

JPG, GIF, PNG
권장 사이즈 128px, 최대 250KB

앱 이름

NodeBird

회사 이름

NodeBird

• 입력된 정보는 사용자가 카카오 로그인할 때 표시됩니다.

• 정보가 정확하지 않은 경우 서비스 이용이 제한될 수 있습니다.

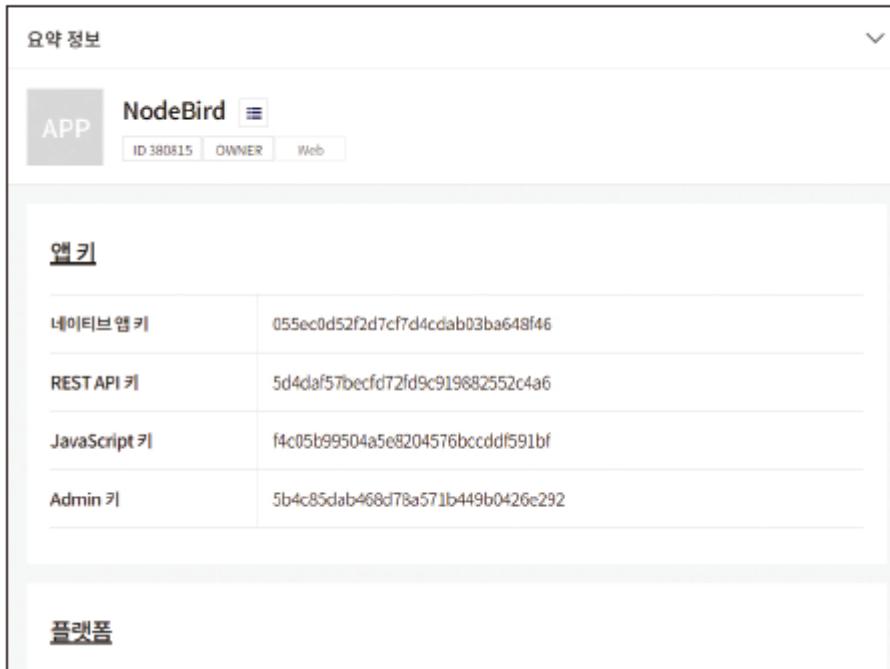
취소

저장

10. 카카오 앱 키 저장하기

» REST API 키를 저장해서 .env에 저장

▼그림 9-12 앱 생성 후 화면



.env

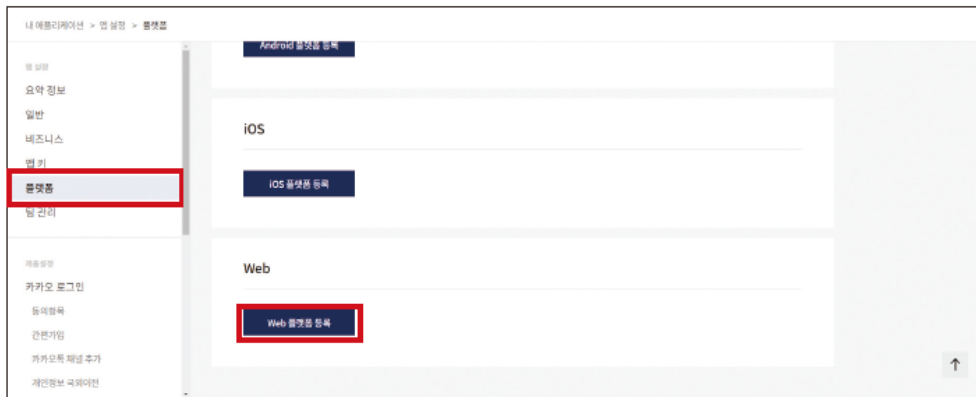
```
COOKIE_SECRET=nodebirdsecret  
KAKAO_ID=5d4daf57becfd72fd9c919882552c4a6
```

11. 카카오 웹 플랫폼 추가

» 웹 플랫폼을 추가해야 callbackURL 등록할 수 있음

- <http://localhost:8001> 등록

▼ 그림 9-13 플랫폼 등록 버튼 클릭



▼ 그림 9-14 사이트 도메인 추가

Web 플랫폼 등록

사이트 도메인

JavaScript SDK, 카카오링크에서 사용됩니다.
여러개의 도메인은 준비공간으로 추가해주세요. 최대 10까지 등록 가능합니다. 추가 등록은 포럼(데브독)으로 문의주세요.

<http://localhost:8001>

취소 저장

Redirect URI

Redirect URI

카카오 로그인에서 사용할 OAuth Redirect URI를 설정합니다.
여러개의 URI를 줄바꿈으로 추가해주세요. (최대 10개)
REST API로 개발하는 경우 필수로 설정해야 합니다.

<http://localhost:8001/auth/kakao/callback>

취소 저장



12. 카카오 동의항목 설정

» 이메일, 생일 등의 정보를 얻기 위해 동의항목 설정

▼ 그림 9-16 로그인 동의항목

동의항목 설정

항목

카카오계정(이메일) / account_email

동의 단계

☐ 필수 동의 (강수 필요)
카카오 로그인 시 사용자가 필수로 동의해야 합니다.

☒ 선택 동의
사용자가 동의하지 않아도 카카오 로그인을 완료할 수 있습니다.

☐ 이용 중 동의
카카오 로그인 시 동의를 받지 않고, 항목이 필요한 시점에 동의를 받습니다.

☐ 사용 연합
사용자에게 권한을 요청하지 않습니다.

카카오 계정으로 정보 수집 후 제공

☒ 사용자에게 값이 없는 경우 카카오 계정 정보 입력을 요청하여 수집

동의 목적

앱 이용

개인자 앱 동의항목 관리 화면내에 입력하는 사실이 실제 서비스 내용과 다를 경우 API 서비스의 거부 사유가 될 수 있습니다.

취소

저장

13. 카카오 로그인 시도

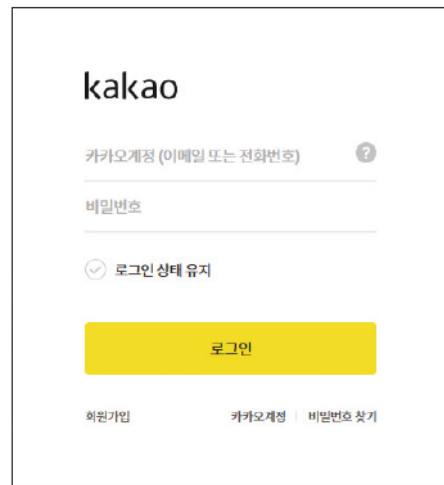
» 카카오톡 로그인 버튼을 누르면 카카오 로그인 창으로 전환

- 계정 동의 후 다시 NodeBird 서비스로 리다이렉트

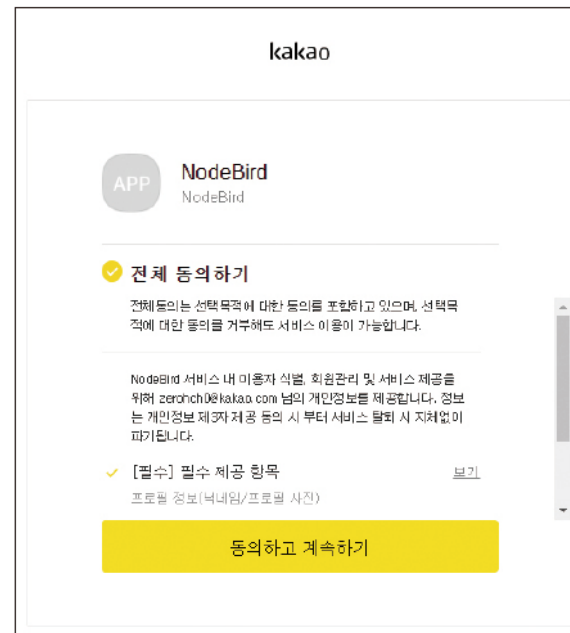
▼ 그림 9-17 카카오톡 버튼 클릭



▼ 그림 9-18 카카오 로그인 화면



▼ 그림 9-19 애플리케이션 제공 항목 동의 화면



9.4 Multer 모듈로 이미지 업로드 구현하기



1. 이미지 업로드 구현

» form 태그의 enctype이 multipart/form-data

- body-parser로는 요청 본문을 해석할 수 없음
- multer 패키지 필요

콘솔

```
$ npm i multer
```

- 이미지를 먼저 업로드하고, 이미지가 저장된 경로를 반환할 것임
- 게시글 form을 submit할 때는 이미지 자체 대신 경로를 전송



2. 이미지 업로드 라우터 구현

» fs.readdir, fs.mkdirSync로 upload 폴더가 없으면 생성

» multer() 함수로 업로드 미들웨어 생성

- storage: diskStorage는 이미지를 서버 디스크에 저장(destination은 저장 경로, filename은 저장 파일명)
- limits는 파일 최대 용량(5MB)
- upload.single('img'): 요청 본문의 img에 담긴 이미지 하나를 읽어 설정대로 저장하는 미들웨어
- 저장된 파일에 대한 정보는 req.file 객체에 담김

routes/post.js

```
const express = require('express');
const multer = require('multer');
const path = require('path');
const fs = require('fs');

const { Post, Hashtag } = require('../models');
const { isLoggedIn } = require('./middlewares');

const router = express.Router();

try {
  fs.readdirSync('uploads');
} catch (error) {
  console.error('uploads 폴더가 없어 uploads 폴더를 생성합니다.');
```

```
  fs.mkdirSync('uploads');
}

const upload = multer({
  storage: multer.diskStorage({
    destination(req, file, cb) {
      cb(null, 'uploads/');
    },
    filename(req, file, cb) {
      const ext = path.extname(file.originalname);
      cb(null, path.basename(file.originalname, ext) + Date.now() + ext);
    },
  }),
  limits: { fileSize: 5 * 1024 * 1024 },
});

router.post('/img', isLoggedIn, upload.single('img'), (req, res) => {
  console.log(req.file);
  res.json({ url: `/img/${req.file.filename}` });
});
```



3. 게시물 등록

» upload2.none()은
multipart/formdata 타입의
요청이지만 이미지는 없을 때 사용

- 게시물 등록 시 아까 받은 이미지
경로 저장
- 게시물에서 해시태그를 찾아서
게시글과 연결(post.addHashtags)
- findOrCreate는 기존에 해시태그가
존재하면 그걸 사용하고, 없다면

▼ 그림 9-21 해시태그와 게시물 연결

```
[ '#노드', '#익스프레스', '#제로초' ]
```

↓ hashtags.map

```
[findOrCreate, findOrCreate, findOrCreate]
```

↓ Promise.all로 모두 실행

```
[[모델, bool], [모델, bool], [모델, bool]]
```

↓ result.map(r => r[0])

```
[모델, 모델, 모델]
```

↓

```
post.addHashtag( )
```

```
const upload2 = multer();
router.post('/', isLoggedIn, upload2.none(), async (req, res, next) => {
  try {
    const post = await Post.create({
      content: req.body.content,
      img: req.body.url,
      UserId: req.user.id,
    });

    const hashtags = req.body.content.match(/#[^\s#]+/g);
    if (hashtags) {
      const result = await Promise.all(
        hashtags.map(tag => {
          return Hashtag.findOrCreate({
            where: { title: tag.slice(1).toLowerCase() },
          })
        })
      );
      await post.addHashtags(result.map(r => r[0]));
    }
    res.redirect('/');
  } catch (error) {
    console.error(error);
    next(error);
  }
});
```

```
module.exports = router;
```



4. 메인 페이지에 게시물 보여주기

» 메인 페이지(/) 요청 시 게시글을 먼저 조회한 후 템플릿 엔진 렌더링

- include로 관계가 있는 모델을 합쳐서 가져올 수 있음
- Post와 User는 관계가 있음 (1대다)
- 게시글을 가져올 때 게시글 작성자까지 같이 가져오는 것)

routes/page.js

```
const express = require('express');
const { isLoggedIn, isNotLoggedIn } = require('./middlewares');
const { Post, User } = require('../models');

const router = express.Router();
...
router.get('/', async (req, res, next) => {
  try {
    const posts = await Post.findAll({
      include: {
        model: User,
        attributes: ['id', 'nick'],
      },
      order: [['createdAt', 'DESC']],
    });
    res.render('main', {
      title: 'NodeBird',
      twits: posts,
    });
  } catch (err) {
    console.error(err);
    next(err);
  }
});

module.exports = router;
```

9.5 프로젝트 마무리하기



1. 팔로잉 기능 구현

» POST /:id/follow 라우터 추가

- /사용자아이디/follow
- 사용자 아이디는 req.params.id로 접근
- user.addFollowing(사용자아이디)로 팔로잉하는 사람 추가

routes/user.js

```
const express = require('express');

const { isLoggedIn } = require('./middlewares');
const User = require('../models/user');

const router = express.Router();

router.post('/:id/follow', isLoggedIn, async (req, res, next) => {
  try {
    const user = await User.findOne({ where: { id: req.user.id } });
    if (user) {
      await user.addFollowing(parseInt(req.params.id, 10));
      res.send('success');
    } else {
      res.status(404).send('no user');
    }
  } catch (error) {
    console.error(error);
    next(error);
  }
});

module.exports = router;
```



2. 팔로잉 기능 구현

» deserializeUser 수정

- req.user.Followers로 팔로워 접근 가능
- req.user.Followings로 팔로잉 접근
- 단, 목록이 유출되면 안 되므로 팔로워/팔로잉 숫자만 프론트로 전달

passport/index.js

```
...
passport.deserializeUser((id, done) => {
  User.findOne({
    where: { id },
    include: [{
      model: User,
      attributes: ['id', 'nick'],
      as: 'Followers',
    }, {
      model: User,
      attributes: ['id', 'nick'],
      as: 'Followings',
    }],
  })
  .then(user => done(null, user))
  .catch(err => done(err));
});
...
```

routes/page.js

```
...
router.use((req, res, next) => {
  res.locals.user = req.user;
  res.locals.followerCount = req.user ? req.user.Followers.length : 0;
  res.locals.followingCount = req.user ? req.user.Followings.length : 0;
  res.locals.followerIdList = req.user ? req.user.Followings.map(f => f.id) : [];
  next();
});
...
```




3. 해시태그 검색 기능 추가

» GET /hashtag 라우터 추가

- 해시태그를 먼저 찾고(hashtag)
- hashtag.getPosts로 해시태그와 관련된 게시글을 모두 찾음
- 찾으면서 include로 게시글 작성자 모델도 같이 가져옴

routes/page.js

```
const express = require('express');
const { isLoggedIn, isNotLoggedIn } = require('./middlewares');
const { Post, User, Hashtag } = require('../models');

const router = express.Router();
...
router.get('/hashtag', async (req, res, next) => {
  const query = req.query.hashtag;
  if (!query) {
    return res.redirect('/');
  }
  try {
    const hashtag = await Hashtag.findOne({ where: { title: query } });
    let posts = [];
    if (hashtag) {
      posts = await hashtag.getPosts({ include: [{ model: User }] });
    }

    return res.render('main', {
      title: `${query} | NodeBird`,
      twits: posts,
    });
  } catch (error) {
    console.error(error);
    return next(error);
  }
});

module.exports = router;
```



4. 업로드한 이미지 제공하기

- » express.static 미들웨어로 uploads 폴더에 저장된 이미지 제공
- 프론트엔드에서는 /img/이미지명 주소로 이미지 접근 가능

app.js

```
...
const pageRouter = require('./routes/page');
const authRouter = require('./routes/auth');
const postRouter = require('./routes/post');
const userRouter = require('./routes/user');
const { sequelize } = require('./models');
const passportConfig = require('./passport');

...
app.use(morgan('dev'));
app.use(express.static(path.join(__dirname, 'public')));
app.use('/img', express.static(path.join(__dirname, 'uploads')));
app.use(express.json());

...
app.use('/', pageRouter);
app.use('/auth', authRouter);
app.use('/post', postRouter);
app.use('/user', userRouter);

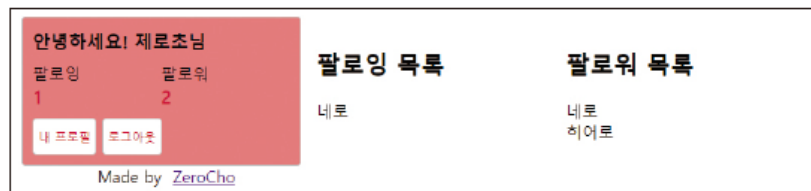
...
```



5. 프로젝트 화면

» 서버를 실행하고 <http://localhost:8001> 접속

▼ 그림 9-22 프로필 페이지 화면



▼ 그림 9-23 해시태그 검색 화면

